An Overview of Big-O

Big-O notation is a mathematical notation that is used to describe the performance or complexity of an algorithm, specifically how long an algorithm takes to run as the input size grows. Understanding Big-O notation is essential for software engineers, as it allows them to analyze and compare the efficiency of different algorithms and make informed decisions about which one to use in a given situation. In this guide, we will cover the basics of Big-O notation and how to use it to analyze the performance of algorithms.

What is Big-O?

Big-O notation is a way of expressing the time (or space) complexity of an algorithm. It provides a rough estimate of how long an algorithm takes to run (or how much memory it uses), based on the size of the input. For example, an algorithm with a time complexity of $O(n)$ means that the running time increases linearly with the size of the input.

What is time complexity?

Time complexity is a measure of how long an algorithm takes to run, based on the size of the input. It is expressed using Big-O notation, which provides a rough estimate of the running time. An algorithm with a lower time complexity will generally be faster than an algorithm with a higher time complexity.

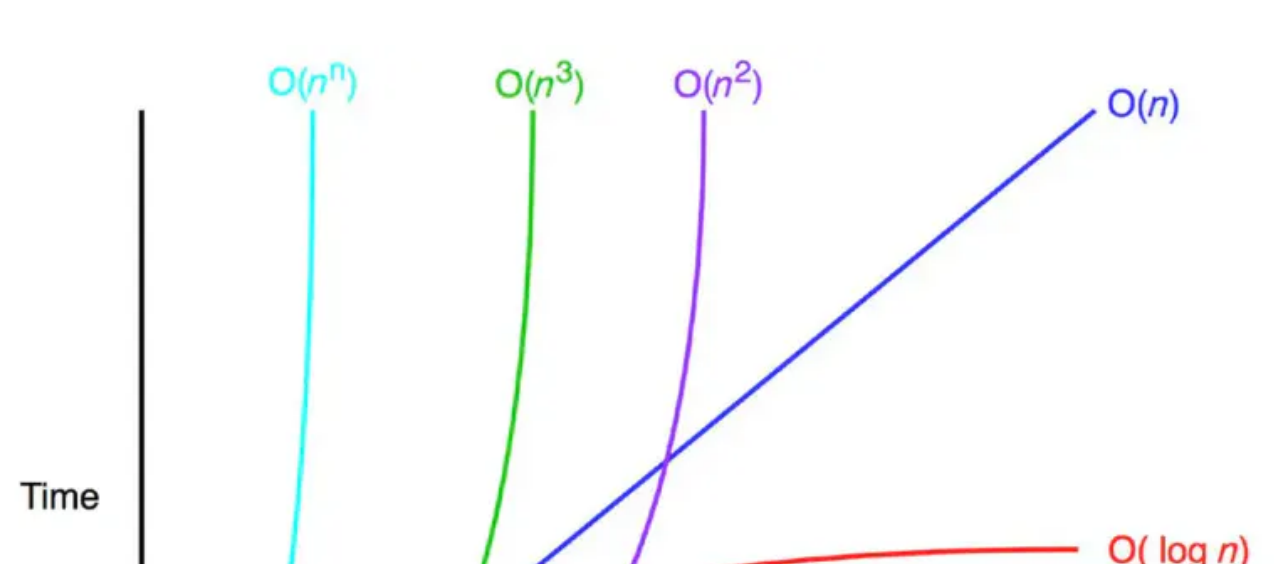
What is space complexity?

Space complexity is a measure of how much memory an algorithm requires, based on the size of the input. Like time complexity, it is expressed using Big-O notation. An algorithm with a lower space complexity will generally require less memory than an algorithm with a higher space complexity.

Examples of time complexity

Here are some examples of how different time complexities are expressed using Big-O notation:

- $O(1)$: Constant time. The running time is independent of the size of the input.
- $O(n)$: Linear time. The running time increases linearly with the size of the input.
- $O(n^2)$: Quadratic time. The running time is proportional to the square of the size of the input.
- $O(logn)$: Logarithmic time. The running time increases logarithmically with the size of the input.
- $O(2^n)$: Exponential time. The running time increases exponentially in the size of the input.

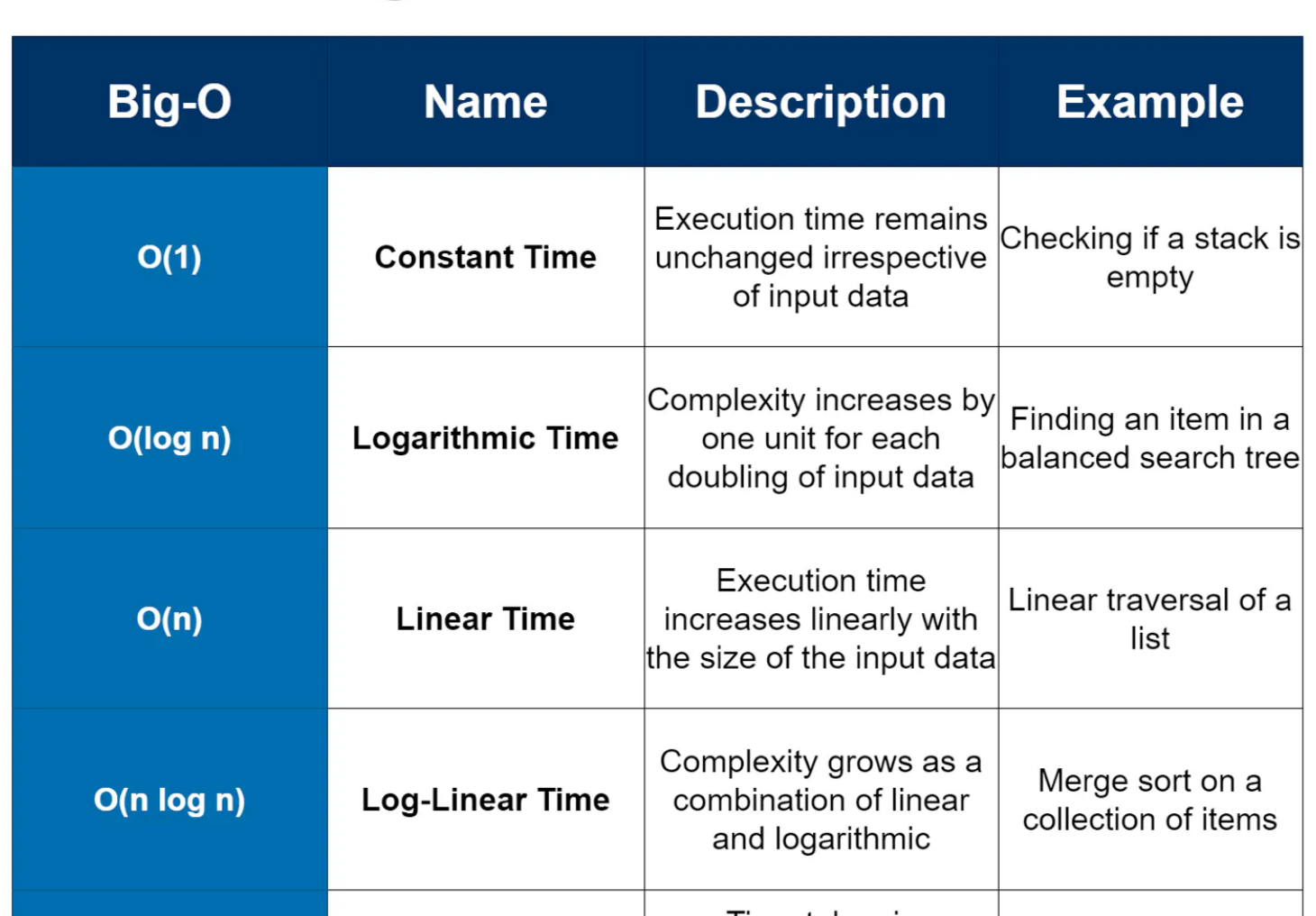


Famous Time Complexities

Big-O notation uses mathematical notation to describe the upper bound of an algorithm's running time. It provides a way to compare the running time of different algorithms, regardless of the specific hardware or implementation being used.

It's important to note that Big-O notation only provides an upper bound on the running time of an algorithm. This means that an algorithm with a time complexity of $O(n)$ could potentially run faster than an algorithm with a time complexity of $O(logn)$ in some cases, depending on the specific implementation and hardware being used.

Additionally, Big-O notation only considers the dominant term in the running time equation. For example, an algorithm with a running time of $O(n^2+n)$ would be simplified to $O(n^2)$.



Big-O Cheat Sheet

Big-O	Name	Description	Example
O(1)	Constant Time	Execution time remains unchanged irrespective of input data	Checking if a stack is empty
O(log n)	Logarithmic Time	Complexity increases by one unit for each doubling of input data	Finding an item in a balanced search tree
O(n)	Linear Time	Execution time increases linearly with the size of the input data	Linear traversal of a list
O(n log n)	Log-Linear Time	Complexity grows as a combination of linear and logarithmic	Merge sort on a collection of items
O(n^2)	Quadratic Time	Time taken is proportional to the square of the number of elements	Checking all possible pairs in an array
O(n^3)	Cubic Time	Execution time is proportional to the cube of the number of elements	Matrix multiplication of n x n matrices
O(2^n)	Exponential Time	Time doubles for every new element added	Generating all subsets of a given set
O(n!)	Factorial Time	Complexity grows factorially based on the size of the dataset	Determining all permutations of a given list

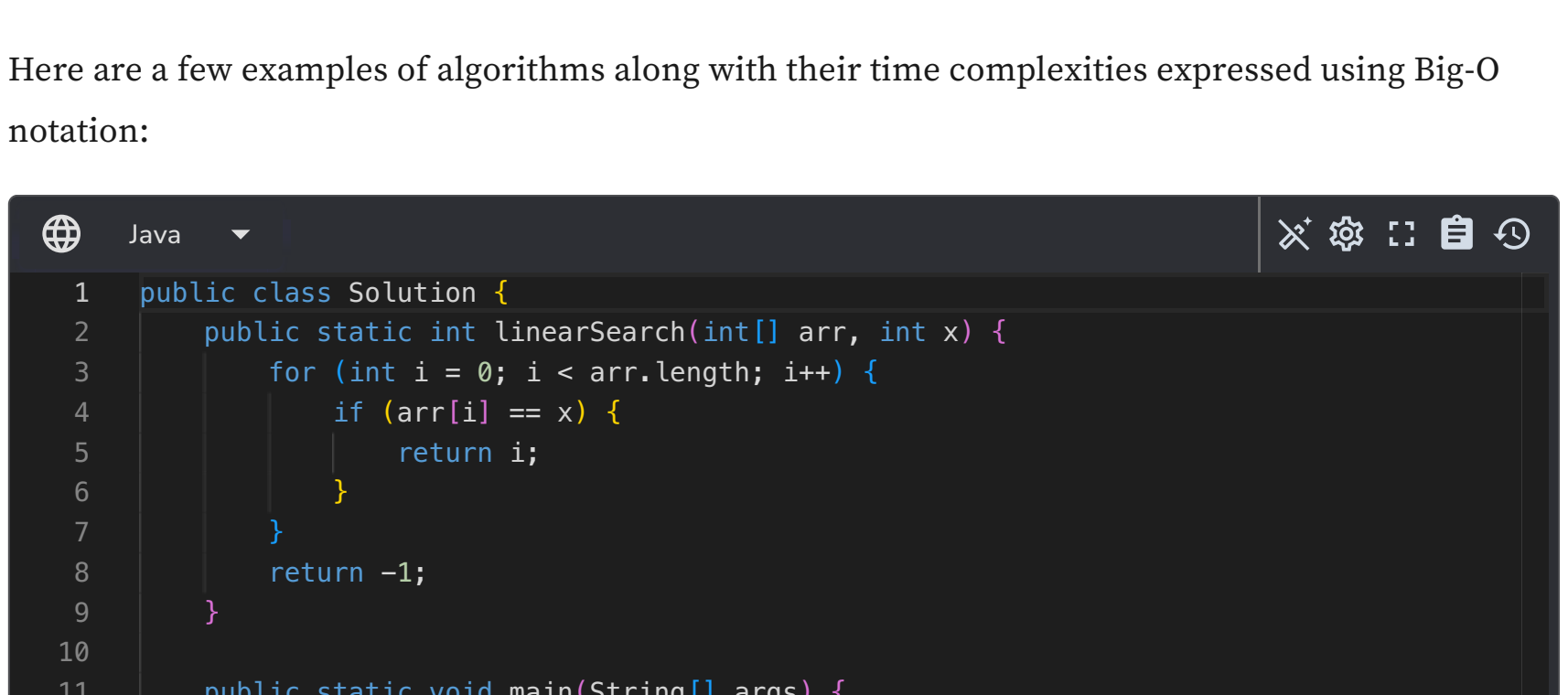
DesignGurus.io

Famous Big-O Notations with Examples.

Example algorithms and their time complexities

1. Linear search-$O(n)$

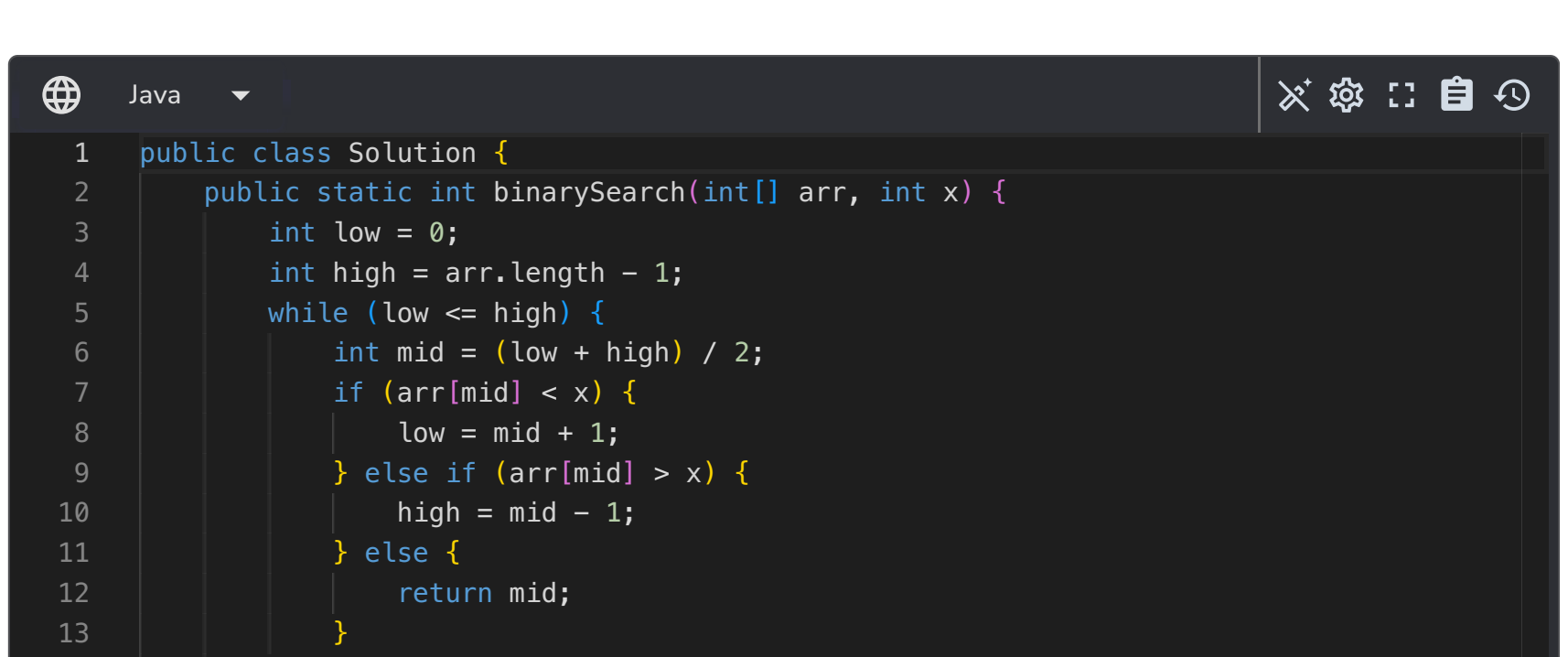
Here are a few examples of algorithms along with their time complexities expressed using Big-O notation:



```java
public class Solution {
    public static int linearSearch(int[] arr, int x) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == x) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {3, 5, 7, 9, 11};
        int x = 7;
        System.out.println(linearSearch(arr, x));
    }
}
```

This linear search algorithm has a time complexity of $O(n)$, since the running time is directly proportional to the size of the input. If the input size is n, the linear search will take n steps to complete.

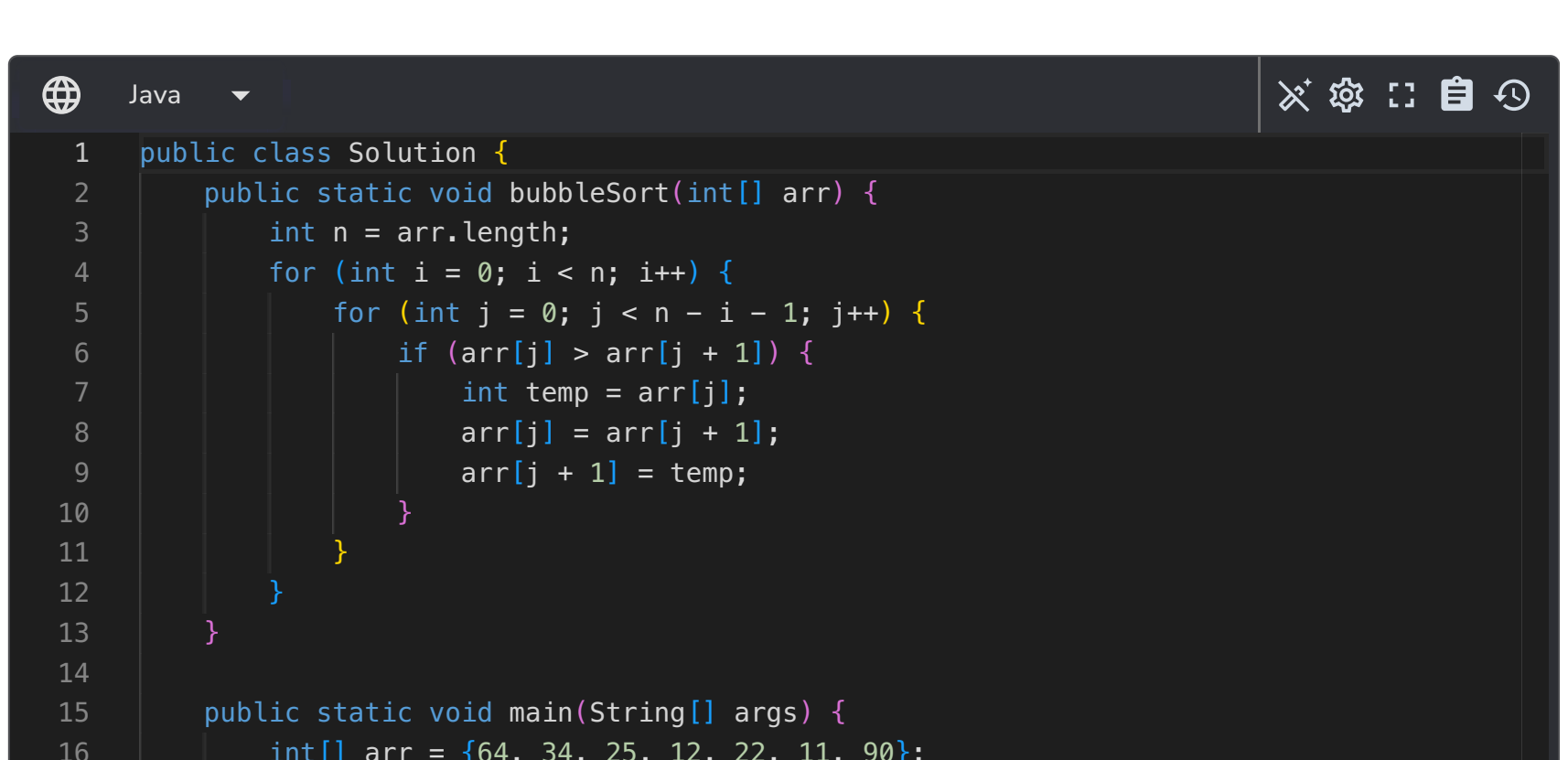
2. Binary search-$O(logn)$



```java
public class Solution {
    public static int binarySearch(int[] arr, int x) {
        int low = 0;
        int high = arr.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] < x) {
                low = mid + 1;
            } else if (arr[mid] > x) {
                high = mid - 1;
            } else {
                return mid;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {2, 3, 4, 10, 40};
        int x = 10;
        System.out.println(binarySearch(arr, x));
```

This binary search algorithm has a time complexity of $O(logn)$, since the running time increases logarithmically with the size of the input. If the input size is n, it will take approximately $log(n)$ steps to complete the binary search.

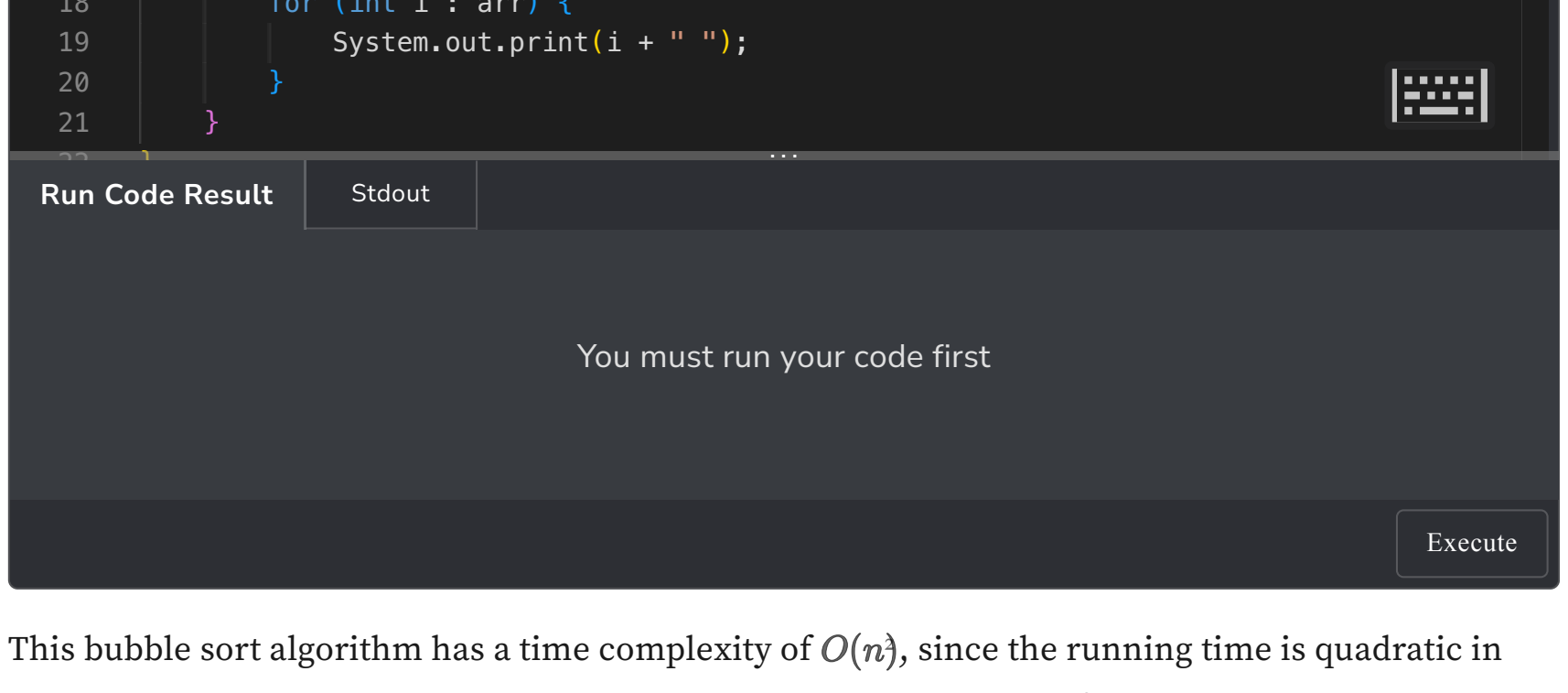
3. Bubble sort-$O(n^2)$



```java
public class Solution {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        for (int i : arr) {
            System.out.print(i + " ");
        }
    }
```

This bubble sort algorithm has a time complexity of $O(n^2)$, since the running time is quadratic in the size of the input. If the input size is n, it will take approximately n^2 steps to complete the bubble sort.

4. Quick Sort-$O(n*logn)$



```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    public static List<Integer> quickSort(List<Integer> arr) {
        if (arr.size() <= 1) {
            return arr;
        }
        int pivot = arr.get(arr.size() / 2);
        List<Integer> left = new ArrayList<>();
        List<Integer> middle = new ArrayList<>();
        List<Integer> right = new ArrayList<>();
        for (int x : arr) {
            if (x < pivot) left.add(x);
            else if (x > pivot) right.add(x);
            else middle.add(x);
        }
        List<Integer> sorted = new ArrayList<>();
        sorted.addAll(quickSort(left));
        sorted.addAll(middle);
        sorted.addAll(quickSort(right));
```

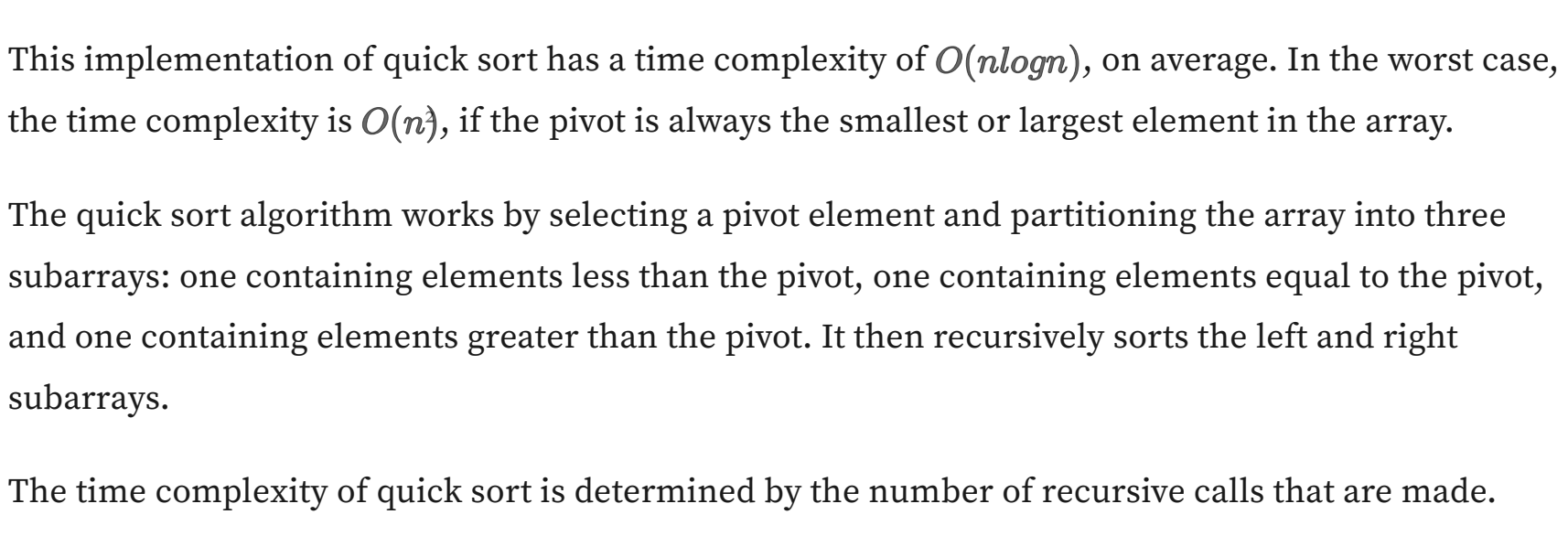
This implementation of quick sort has a time complexity of $O(nlogn)$, on average. In the worst case, the time complexity is $O(n^2)$, if the pivot is always the smallest or largest element in the array.

The quick sort algorithm works by selecting a pivot element and partitioning the array into three subarrays: one containing elements less than the pivot, one containing elements equal to the pivot, and one containing elements greater than the pivot. It then recursively sorts the left and right subarrays.

The time complexity of quick sort is determined by the number of recursive calls that are made. Each recursive call processes approximately $n/2$ elements, so the total number of recursive calls is approximately $log(n)$. The running time of each recursive call is $O(n)$, so the total running time is $O(nlogn)$.

5. Fibonacci sequence-$O(2^n)$

Here is an implementation of an algorithm for calculating the n'th element of the Fibonacci sequence:



```java
public class Solution {
    public static int fibonacci(int n) {
        if (n == 0) {
            return 0;
        } else if (n == 1) {
            return 1;
        } else {
            return fibonacci(n - 1) + fibonacci(n - 2);
        }
    }

    public static void main(String[] args) {
        int n = 10;  // Example input
        System.out.println("Fibonacci number at position " + n + " is " + fibonacci(n));
    }
}
```

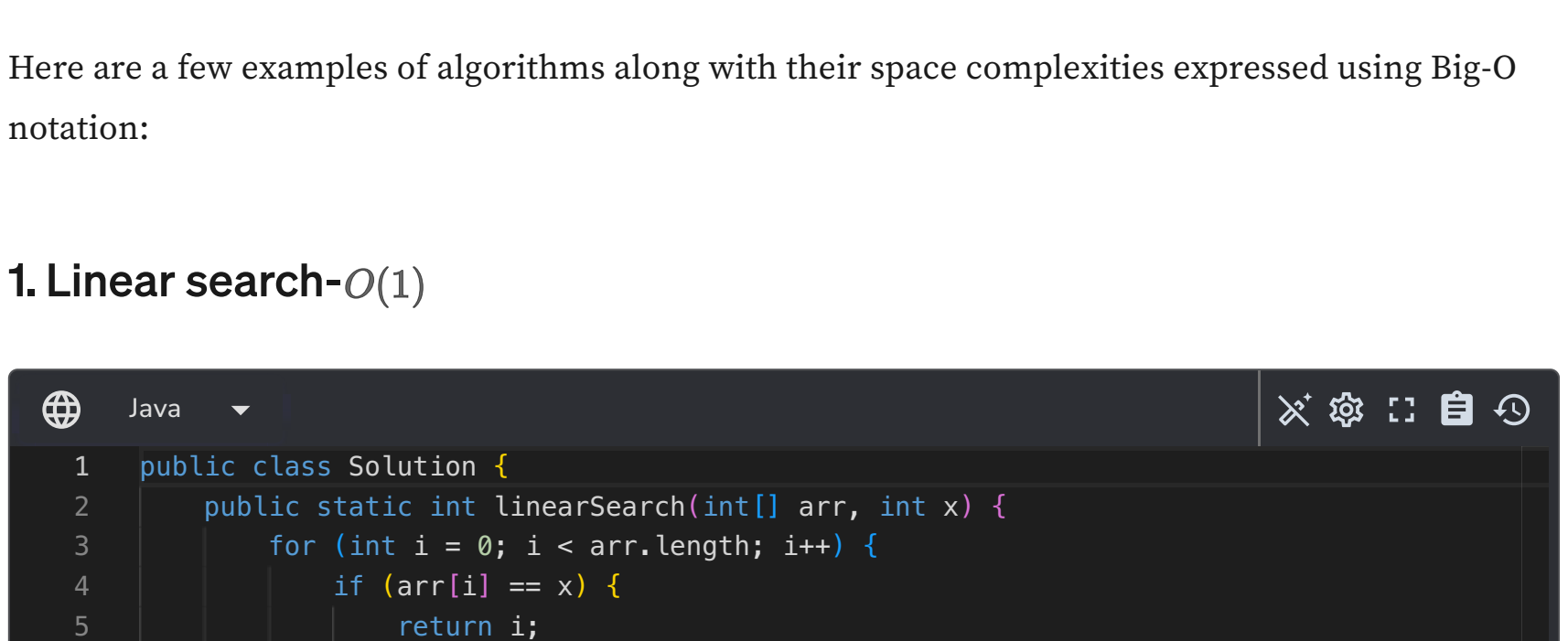
This implementation of the Fibonacci sequence has a time complexity of $O(2^n)$, which is exponential in the size of the input. This is because each call to fibonacci(n) results in two additional recursive calls to `fibonacci(n-1)` and `fibonacci(n-2)`.

The time complexity of this algorithm is determined by the number of recursive calls that are made. Each recursive call processes one element, so the total number of recursive calls is approximately 2^n. The running time of each recursive call is $O(1)$, so the total running time is $O(2^n)$.

Example algorithms and their space complexities

Here are a few examples of algorithms along with their space complexities expressed using Big-O notation:

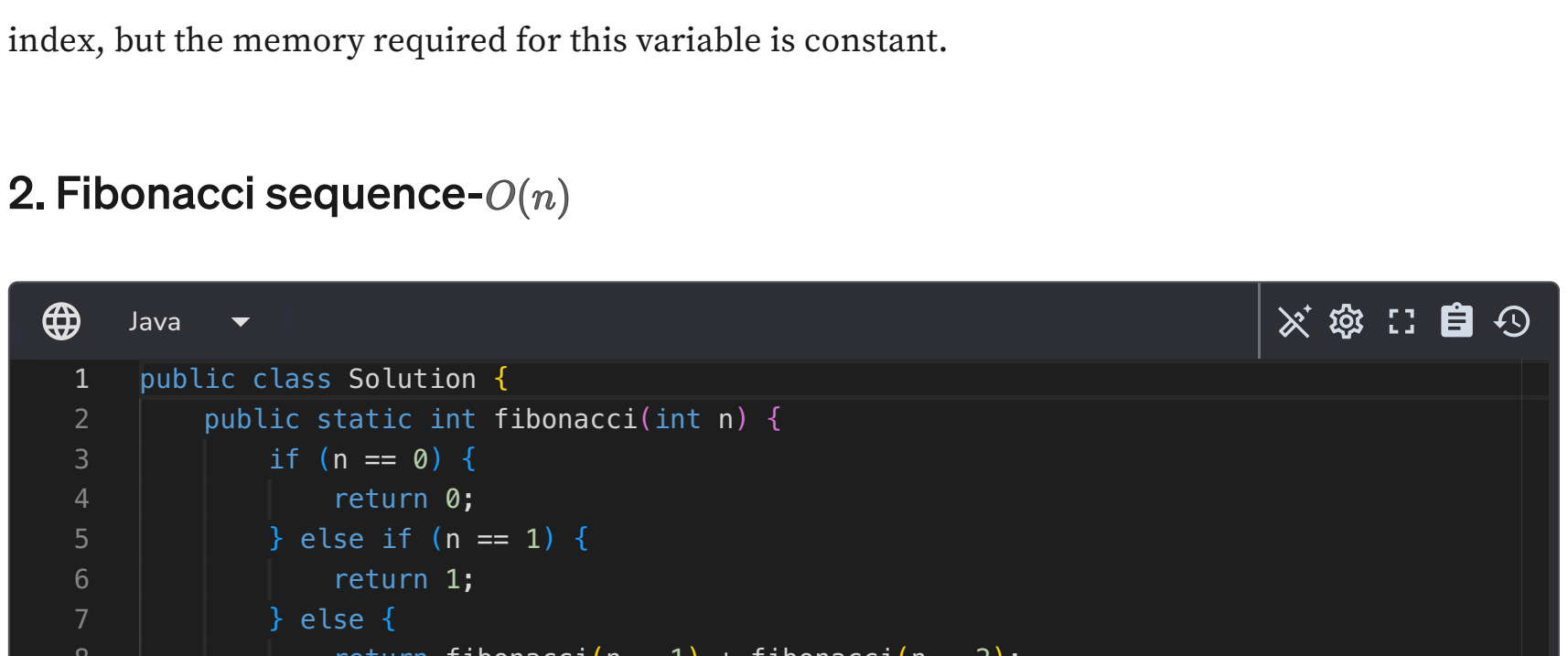
1. Linear search-$O(1)$



```java
public class Solution {
    public static int linearSearch(int[] arr, int x) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == x) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {3, 4, 2, 7, 9};
        int x = 7;
        int result = linearSearch(arr, x);
        System.out.println(result != -1 ? "Element found at index " + result : "Element
    }
}
```

This linear search algorithm has a space complexity of O(1), since it only requires a constant amount of memory regardless of the size of the input. It creates a single variable (i) to store the loop index, but the memory required for this variable is constant.

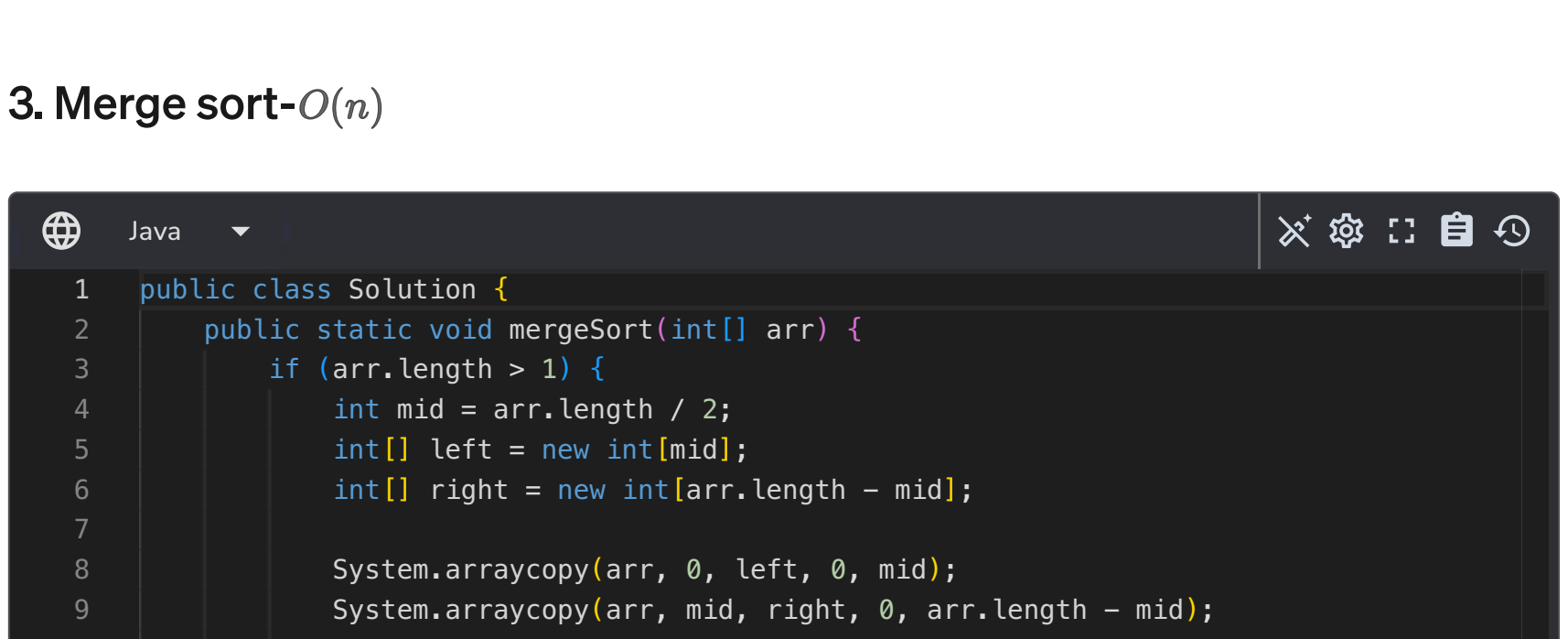
2. Fibonacci sequence-$O(n)$



```java
public class Solution {
    public static int fibonacci(int n) {
        if (n == 0) {
            return 0;
        } else if (n == 1) {
            return 1;
        } else {
            return fibonacci(n - 1) + fibonacci(n - 2);
        }
    }

    public static void main(String[] args) {
        int n = 10;  // Example input
        System.out.println("Fibonacci number at position " + n + " is " + fibonacci(n));
    }
}
```

This algorithm for calculating the nth element of the Fibonacci sequence has a space complexity of $O(n)$, since it uses recursion and the call stack grows with the size of the input. Each recursive call requires additional memory to store the state of the function, and the total amount of memory required increases linearly with the size of the input.

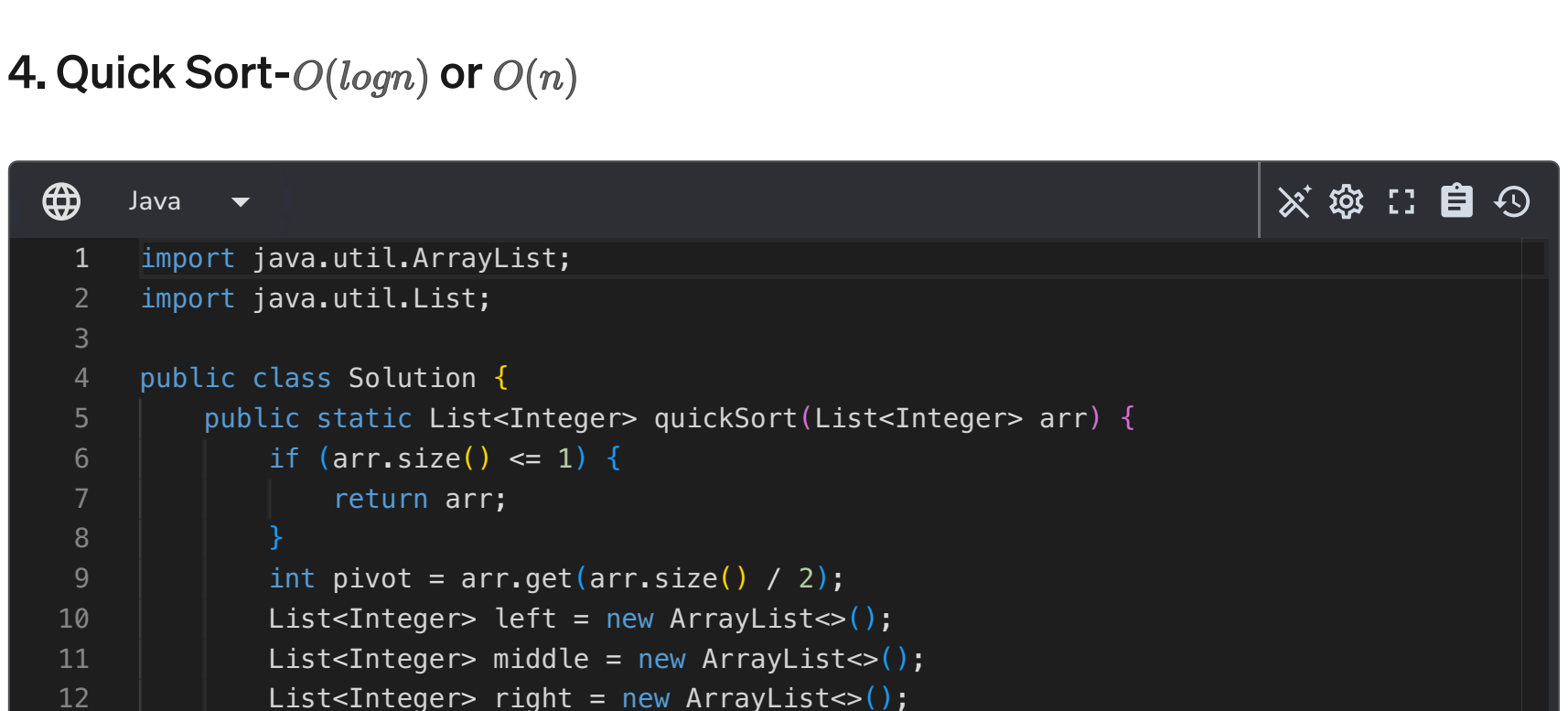
3. Merge sort-$O(n)$



```java
public class Solution {
    public static void mergeSort(int[] arr) {
        if (arr.length > 1) {
            int mid = arr.length / 2;
            int[] left = new int[mid];
            int[] right = new int[arr.length - mid];

            System.arraycopy(arr, 0, left, 0, mid);
            System.arraycopy(arr, mid, right, 0, arr.length - mid);

            mergeSort(left);
            mergeSort(right);

            int i = 0, j = 0, k = 0;

            while (i < left.length && j < right.length) {
                if (left[i] < right[j]) {
                    arr[k++] = left[i++];
                } else {
                    arr[k++] = right[j++];
                }
```

This merge sort algorithm has a space complexity of $O(n)$, since it creates two additional arrays (left and right) to store the left and right halves of the input during each recursive call. The memory required to store these arrays increases linearly with the size of the input.

4. Quick Sort-$O(logn)$ or $O(n)$

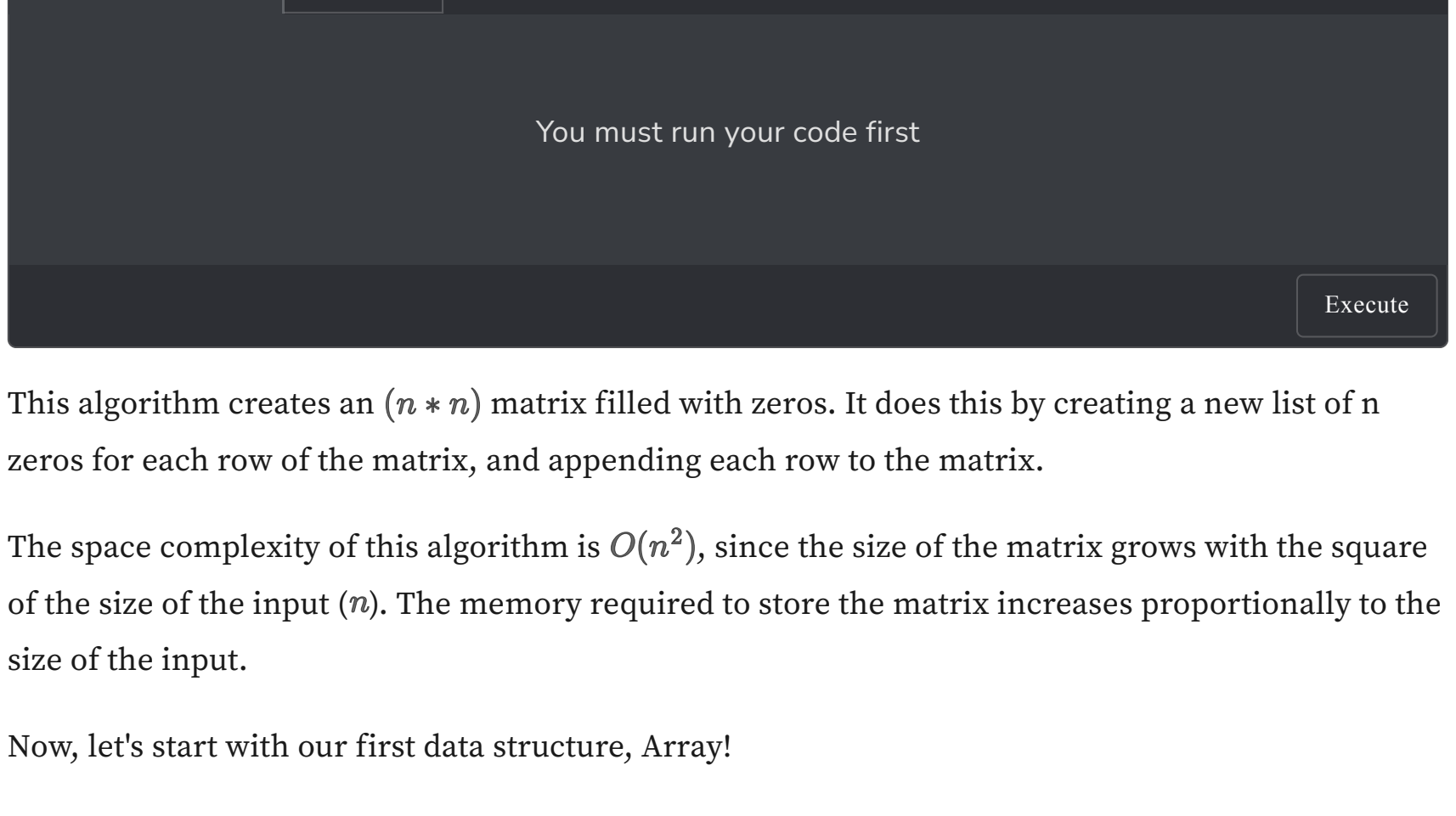


```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    public static List<Integer> quickSort(List<Integer> arr) {
        if (arr.size() <= 1) {
            return arr;
        }
        int pivot = arr.get(arr.size() / 2);
        List<Integer> left = new ArrayList<>();
        List<Integer> middle = new ArrayList<>();
        List<Integer> right = new ArrayList<>();
        for (int x : arr) {
            if (x < pivot) left.add(x);
            else if (x > pivot) right.add(x);
            else middle.add(x);
        }
        List<Integer> sorted = new ArrayList<>();
        sorted.addAll(quickSort(left));
        sorted.addAll(middle);
        sorted.addAll(quickSort(right));
```

The quick sort algorithm works by selecting a pivot element and partitioning the array into three subarrays: one containing elements less than the pivot, one containing elements equal to the pivot, and one containing elements greater than the pivot. It then recursively sorts the left and right subarrays.

Quick sort has a space complexity of $O(logn)$, on average. In the worst case, the space complexity is $O(n)$, if the pivot is always the smallest or largest element in the array. This is because the call stack grows with the number of recursive calls, and the number of recursive calls is logarithmic in the size of the input.

5. Create Matrix-$O(n^2)$



```java
public class Solution {
    public static int[][] createMatrix(int n) {
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = 0;
            }
        }
        return matrix;
    }

    public static void main(String[] args) {
        int n = 3;  // Example size
        int[][] matrix = createMatrix(n);
        for (int[] row : matrix) {
            for (int val : row) {
                System.out.print(val + " ");
            }
            System.out.println();
        }
    }
```

This algorithm creates an $(n*n)$ matrix filled with zeros. It does this by creating a new list of n zeros for each row of the matrix, and appending each row to the matrix.

The space complexity of this algorithm is $O(n^2)$, since the size of the matrix grows with the square of the size of the input (n). The memory required to store the matrix increases proportionally to the size of the input.

Now, let's start with our first data structure, Array!

← Previous
Types Of Data Structures
Next →
Introduction To Arrays
☐ Mark as Completed